

Dự án Quý 1: RetailAnalytics Data Lake (Medallion Architecture)

Tên dự án: Retail Sales Modern Data Lakehouse Foundation

Mô tả: Xây dựng nền tảng Data Lake sử dụng **Medallion Architecture** (Bronze → Silver → Gold) cho chuỗi bán lẻ. Toàn bộ pipeline được triển khai bằng **Python**, dữ liệu được lưu dưới dạng **Parquet** (có partitioning), chuẩn bị sẵn sàng cho Iceberg/Delta Lake sau này.

Mục tiêu Quý 1:

- Thành thạo Python production-grade cho Data Engineering.
- Hiểu sâu Medallion Architecture trên Data Lake.
- Xây dựng ingestion & transformation framework có thể tái sử dụng.
- Áp dụng đầy đủ Software Engineering best practices.

Tech Stack Quý 1 (Data Lake oriented)

- **Language:** Python 3.11+
- **Data Processing:** Polars (chính) + PyArrow
- **Storage:**
 - Local filesystem (dùng partitioning)
 - **MinIO** (tùy chọn, giả lập S3)
- **Query Engine:** **DuckDB** (rất mạnh cho Gold layer và analytics)
- **Packaging:** Poetry (hoặc uv)
- **Logging:** structlog (JSON)
- **Config:** pydantic-settings
- **Testing:** pytest + Great Expectations (light)
- **Container:** Docker (multi-stage)
- **CI/CD:** GitHub Actions
- **Version Control:** Git (trunk-based)

Kiến trúc Tổng thể (Medallion trên Data Lake)

text

Raw Sources (CSV files)

↓

Bronze Layer → Parquet raw + metadata (partition by year/month/day)

↓

Silver Layer → Cleaned, standardized, validated Parquet

↓

Gold Layer → Star/Snowflake schema (Parquet hoặc DuckDB views/tables)

+ SCD + Aggregations

Chi tiết các Layer

1. Bronze Layer – Raw Ingestion

Mục tiêu: Đưa dữ liệu thô vào Data Lake mà **không biến đổi giá trị**.

Công việc chính:

- Đọc 3 nguồn CSV (sales, customer, product)
- Thêm metadata: `_ingested_at`, `_source_file`, `_load_id`, `_ingestion_status`
- Lưu dưới dạng **Parquet** với **partitioning** (ví dụ: `year=2026/month=05/day=26/`)
- Idempotency: Kiểm tra `_load_id` trước khi ghi
- Dùng Polars để đọc nhanh và ép kiểu an toàn (to string ở Bronze)

Kỹ năng áp dụng:

- Type hints, Logging, Retry (tenacity), Profiling, Config management, Unit tests
-

2. Silver Layer – Cleaning & Standardization

Mục tiêu: Tạo phiên bản dữ liệu sạch, đáng tin cậy.

Công việc chính:

- Đọc từ Bronze Parquet
- Cleaning: trim space, lower/upper case, fix encoding
- Standardization: datetime, numeric types, email/phone format
- Validation: business rules, data quality checks
- Deduplication theo business key
- Rejected records → lưu riêng bảng `silver_rejected`
- Lưu kết quả thành Parquet (có partitioning)

Công cụ: Polars (rất mạnh và nhanh)

3. Gold Layer – Dimensional Modeling

Mục tiêu: Xây dựng mô hình phục vụ analytics.

Công việc chính:

- Xây dựng **Star Schema** (fact_sales + dim_customer, dim_product, dim_date, dim_store)
- Triển khai **SCD Type 2** cho Customer & Product
- Tạo các aggregation views
- Sử dụng **DuckDB** để tạo tables/views hoặc materialized Parquet
- Advanced SQL: Window functions, CTEs, Query optimization (DuckDB hỗ trợ rất tốt)

Lợi ích: Vừa dùng Python làm pipeline, vừa dùng SQL mạnh mẽ ở Gold layer.

Engineer Mindset & Best Practices (Áp dụng xuyên suốt)

- **Idempotency & Incremental Load**
 - **Resilience** (retry, error handling, circuit breaker pattern)
 - **Observability** (structlog + metrics: rows in/out, reject rate, execution time)
 - **Data Quality:** Light checks ở Bronze, Heavy checks ở Silver (Great Expectations)
 - **Testing:** Unit test, Integration test (Bronze → Silver → Gold), Data Quality tests
 - **Docker:** Multi-stage Dockerfile cho pipeline
 - **CI/CD:** Lint → Test → Build Docker image
 - **Documentation:** README chuyên nghiệp + Architecture diagram + Data Dictionary
-

Project Deliverables (Portfolio #1)

1. GitHub repo sạch sẽ với cấu trúc rõ ràng
2. Full pipeline: make ingest-bronze, make transform-silver, make build-gold
3. README.md chi tiết (architecture, how to run, tech decisions, challenges)
4. Data Quality Report + Observability logs
5. Advanced SQL queries trên Gold layer (Window functions, SCD logic...)
6. Docker + GitHub Actions demo
7. ER Diagram (Star Schema)